

New Genetic Crossover Operator for the TSP

Sang-Moon Soak¹ and Byung-Ha Ahn¹

Kwang-Ju Institute of Science and Technology, Dept. of Mechatronics,

1 Oryong-dong, Buk-gu, Gwangju, Republic of Korea

{soakbong,bayhay}@kjist.ac.kr

Abstract. Genetic algorithm is very useful method for global search of large search space and has been applied to various problems. It has two kinds of important search mechanisms, crossover and mutation. Especially many researchers have more interested in crossover operator than mutation operator because crossover operator has charge of the responsibility of local search. In this paper we introduce a new crossover operator avoiding the drawback of conventional crossovers. We compare it to several crossover operators for travelling salesman problem (TSP) for showing the performance of proposed crossover.

1 Introduction

Since Holland introduced the Genetic Algorithm (GA) in the early of 1970's, many researchers have become interested in it as a new method for solving large and difficult problems in search and optimization [1]. And for a long time, a large number of operators have been developed for improving the performance of GA because the performance of algorithm depends on an ability of these operators. Especially many researchers have more interest in crossover operator than mutation operator. Because it is the important element that performs the exchange of information between individuals during the generation and global optimal solution is generally known as existing near sub-optimal solution [9], more crossover operators have been developed.

In this paper, we deal with only crossover operators, especially for solving travelling salesman problem (TSP). The TSP is a classical NP-complete problem, which has extremely large search spaces and is very difficult to solve. Since in the early of using GA for solving TSP, just many kinds of permutation crossover operators were introduced such as Cycle crossover (CX), Order Crossover (OX), Position-Based Crossover (PBX), Order-Based Crossover (OBX), Partial-Mapped Crossover (PMX) and so on. Recently, researcher's interest has been changed more and more in direction of using information of parents such as sub-tour [3], [4], [7], edge [3], [5], [10] or edge and distance [2], [6], [8] information of parents. In this paper we propose new subtour-based crossover operator to show that adjacency information of parents may be important and should be preserved and it can be more performed than other operators, and compare with both some permutation crossovers and edge or subtour-based crossovers.

2 Subtour Preservation Crossover (SPX)

The proposed crossover, Subtour Preservation Crossover (SPX), is motivated by various features of edge-based crossover and subtour-based crossover. SPX uses a similar subtour enumeration method to other subtour-based crossovers but it has an amount of difference in method that generates a valid tour. Subtour Exchange Crossover (SXX) [7] and Complete Subtour Exchange Crossover (CSEX) [4] generate simply several offsprings through a process that exchanges and inverts each other common subours in parents and then choose two tours producing the shortest distance in offspring pool as offspring and Distance Preserving Crossover (DPX) [12] uses a greedy reconnection procedure which reconnects the nearest available neighbor among the endpoints of subtours. Otherwise, SPX generates offsprings using common subtour which parents share and edges included in each parents. That is, the method uses maximum information involved in parents.

All the subtour-based crossovers have a severe drawback. If the same parents are selected for crossover, any subtour-based crossovers do not generate offsprings different with parents, so they cannot improve the performance of current solution and should depend on mutation operator for improvement of solution. It is the reason that offsprings are exactly identical with their parents because the subtour itself is offspring. However, as generation increases more and more, the number of same solution in population will be exponentially increased by a selection strategy. So the improvement of solutions will be very difficult by degrees. But in case of SPX, we can solve this problem as introducing new predominant solutions instead of parents. We will explain more detail later. And if SXX and CSEX find a subtour of cities with the same order such as $Subtour_1(1, 2, 3, 4)$ and $Subtour_2(1, 2, 3, 4)$, they generate the same offsprings as current parents. So a supplementary method for solving this problem is very needed. But, although SPX finds the same subtour, it can generate different offsprings provided that selects different starting points at each process reconnecting subtours and isolated cities. Also, subtour-based crossover operators use only subtour among an amount of information of parents augmented over thousands of generations. That is, they are only different from permutation-based crossover operators in that point. The remaining process is very similar to permutation process. So other parental information besides subtour is never considered. But, as mentioned above, proposed method is a crossover operator using amount of parental information, not only common subtour but edges involved in each parent.

2.1 Subtour Analysis

First, we investigate how many identical parents will be selected over generation to generation. It is very important measure because if so, subtour-based crossover operators will be for nothing in the performance. Fig. 1 shows the average number of times selected two identical parents. After 200 generations, the average number of times selected two identical parents is over 20. That is, it shows the identical parents are selected about over 30% for total crossover process ($P_c = 0.6$). So

if we do not consider a supplementary method for avoiding this problem, the improvement of solutions will be more and more difficult.

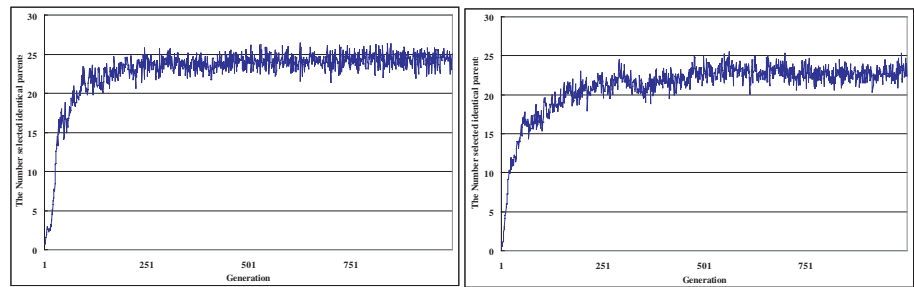


Fig.1. The average number of times selected identical parents : (a) Instance (left) = kroA100, (b) Instance (right) = kroB200 ($pop_size = 200$, $P_c = 0.6$, *Tournament selection*, *Total generation* = 1000, *Iteration* = 30)

2.2 New Offspring

If the identical parents are selected for crossover, SPX generates new offsprings using a local search method. If offsprings produced by simple permutation would be introduced in population, they cannot survive in population filled with solutions holding already good fitness values. So if identical parents are selected in SPX, it first generates two solutions randomly and then apply 2-opt algorithm to each solution for competing with good solution existing already in population. The solutions generated by 2-opt algorithm are local minimum solution. So it will be able to compete with solutions in population.

2.3 Outline of SPX

The procedure of SPX is divided into two steps. The first step is to enumerate common subtours and next is to reconnect each subtour. We will explain more detail using an example. Fig. 2 shows an example of SPX. (a) and (b) is two parents selected respectively, (c) is an overlapped tour of parent tours and (d) shows a common subtours. And (e) shows three offsprings generated by various random selection of starting point and the process of connecting subtours and isolated points. At (d), total 15 points (the end points (8) of each subtours and isolated nodes (7)) can be selected as starting points. In here, note the end points of each subtours and isolated nodes have two possible edges derived from parent. So, if a point is selected as a starting point, shorter edge between two possible edges is selected. If shorter edge was already selected, another edge is selected. And if both edges were already selected at previous step, random selection is performed among endpoints not selected at previous step. If the edge selected has

the length of subtour more than 1 (not an isolated node), all elements within the subtour are connected in turn. This process is continued until a tour is formed. As shown in this figure, SPX can generate various offsprings with preserving common dominant features of parent generation.

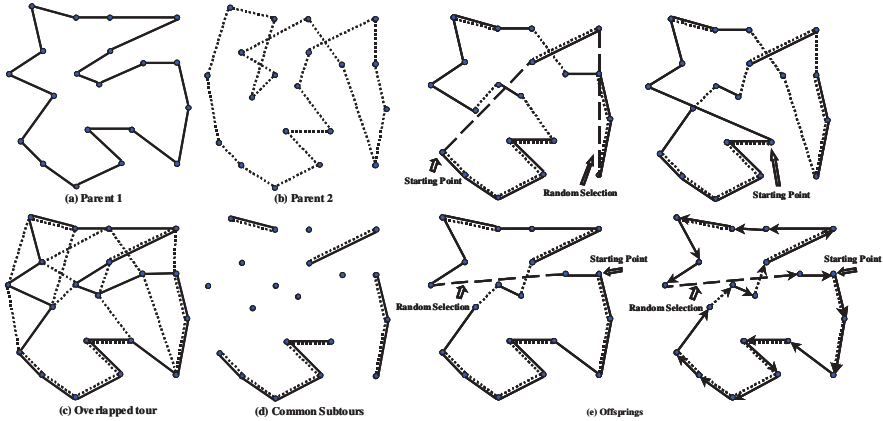


Fig. 2. Example of Subtour Preservation Crossover

3 Experimental Results

We performed an experiment for comparing proposed crossover operator with several crossover operators and all algorithms are implemented in Visual C++ and tested on Pentium IV 1.6Ghz personal computer. All the test problems was obtained from TSPLIB [13]. In order to demonstrate the superiority of SPX, we compared SPX's performance with those of the most outstanding crossover. With regards to compare each crossover operator in justice, we used simple GA. And initial individuals are randomly generated and used tournament strategy and elitist selection rule for the selection. The number of preserved best tour is 3. The mutation operator used in this experiment is inversion operator. The population size (pop_size), crossover probability (P_c) and mutation probability (P_m) are 200, 0.6 and 0.4 respectively. The termination condition is satisfied either when the optimal solution is found or when the best solution of the population remains unchanged for 1000 generations. The experiment was carried out 30 runs on each operator by changing the seed of the random number. The number of offspring generated by SPX sets 5. In this experiment, the quality is defined as $quality = \frac{Fitness - Optimal}{Optimal} \times 100(\%)$, i.e. the quality indicates the percentage over the optimal value.

Table. 1 shows the simulation results of each operator. * indicates the case of $pop_size = 100$. SPX was needed much computation time in comparison to

Table 1. Experiment Results

eil51(426)						rat99(1211)					
	Min	Avg.(Quality)	Max	Gen.	Time(sec.)	Min	Avg.(Quality)	Max	Gen.	Time(sec.)	
MPX	432	440.2(3.3)	450	2019.2	22.7	1286	1334.6(10.2)	1382	2396.1	85.6	
SCH.	430	448.8(5.4)	464	1520.5	6.6	1260	1337.3(10.4)	1431	1947	20.6	
ROX	427	437.9(2.8)	451	1734.8	5.7	1249	1289.6(6.5)	1332	2631	19.3	
HX	426	439.2(3.1)	453	1743.3	8.3	1240	1300.0(7.4)	1382	2135.4	24.7	
GX	431	443.9(4.2)	466	1414.4	6.7	1264	1333.7(10.1)	1430	1643.2	19.2	
ER	430	443(3.9)	459	1794.3	12.6	1285	1329.9(9.8)	1394	2172.9	38.4	
EER	433	448.3(5.2)	464	1506.6	9.7	1282	1339.8(10.6)	1432	1744.2	28.9	
SXX	438	449.4(5.5)	470	1638.5	15.5	1280	1341.1(10.7)	1403	1863.1	41.8	
CSEX	433	447.8(5.1)	471	1500.5	15.3	1286	1340.1(10.6)	1412	1930.9	45.7	
DPX	428	433.8(1.8)	443	1225.2	8.9	1213	1250.9(3.3)	1287	1384.0	47.6	
SPX	426	429.0(0.7)	435	1321.9	35.1	1212	1227.4(1.3)	1283	1518.7	142.9	
*	426	430(0.9)	437	1546.5	23.1	1212	1228.1(1.4)	1283	1798.3	94.1	

kroA(21282)						rat195(2323)					
	Min	Avg.(Quality)	Max	Gen.	Time(sec.)	Min	Avg.(Quality)	Max	Gen.	Time(sec.)	
MPX	21552	22683.0(6.6)	23779	2482.5	90.1	2551	2623.0(12.9)	2697	4089	528.5	
SCH.	22026	23073.9(8.4)	25262	1766.6	18.7	2577	2638.7(13.6)	2761	3474.6	106.8	
ROX	21315	22528.2(5.8)	23947	2525.6	18.5	2466	2532.7(9.0)	2617	5315.5	106.7	
HX	21391	22258.9(4.6)	23285	1837	21.6	2483	2593.7(11.6)	2707	3064.7	105.9	
GX	21668	22755.2(6.9)	24865	1685.1	19.7	2524	2627.4(13.1)	2725	3051	104.6	
ER	21785	22939.4(7.8)	24355	1815.8	32.5	2548	2625(13.0)	2727	3371.6	175	
EER	21638	23115.0(8.6)	24837	1686.3	28.1	2526	2633.9(13.4)	2737	3218.6	159.2	
SXX	21807	22775.7(7.0)	23954	1989.8	44.3	2522	2635.3(13.4)	2715	3618.3	409	
CSEX	21721	22919.5(7.7)	24945	1998.4	47.3	2558	2642.9(13.8)	2717	4142.9	490	
DPX	21292	21940.5(3.1)	23059	1292.6	48.1	2370	2400.9(3.4)	2471	1818.5	357.8	
SPX	21282	21445.1(0.8)	21841	1412.5	140.1	2334	2378.2(2.4)	2466	2746.5	1120.0	
*	21282	21418.3(0.6)	21841	1632.5	91.7	2345	2385.3(2.7)	2465	3401.3	701	

Note: () : optimal solution; Gen. : Average Generation; Time : Average CPU Time; * : *pop_size* = 100

other operators but the performance was much superior to the others in all the cases. At the instance eil51 and kroA100, SPX found optimal solution, on the other hand other operators could not find. The proposed operators generated the quality not to excess 3% from optimal solution in all the case. The reason much computation time is needed is for selection of identical parents and this case is increased more and more. So for decreasing computation time, we cut down population size as 100 but the performance was not changed nearly. Especially, SPX indicates better results than DPX, which is known as one of the most effective and predominant operators. It is very encouraging fact.

4 Conclusion

In this paper, we proposed a highly efficient subtour-based crossover, subtour preservation crossover (SPX) for TSP, and experimented the features and performance of SPX. and pointed out the necessity of a supplementary method in subtour-based crossovers. By means of several computational experiments, it has been confirmed that SPX can get better results than permutation based crossovers and other operators using parental information. Although the proposed new crossover operators needed more computation time than others, performance is superior to them. As future work we plan to investigate more powerful local search algorithms such as Lin-Kernighan algorithm (LN) and develop a predo-

minant hybrid genetic algorithm. Because the objective of this paper is to show that the more parental information is used, the better performance can be obtained, we use small size instances. But, for the effectiveness of the SPX, we will apply it to more large problems.

References

1. D. Goldberg : Genetic Algorithms in Search, Optimization, and Machine Learning. Addison Wesley, Reading, MA (1989).
2. J. Grefenstette, R. Gopal, B. Rosmaita, D. Gucht : Genetic Algorithms for the Traveling Salesman Problem. First Int. Conf. on GA & App's, (1985) 160-168.
3. K. Maekawa, N. Mori, H. Tamaki, H. Kita, Y. Nishikawa : A Genetic Solution for the Traveling Salesman Problem by Means of a Thermodynamical Selection Rule. Proc. of IEEE Int. Conf. on EC, (1996) 529-534.
4. K. Katayama, H. Hirabayashi, H. Narihisa : Performance Analysis of a New Genetic Cross-over for the Traveling Salesman Problem. IEICE TRANS. FUNDAMENTALS, VOL. E81-A, NO. 5 (1998) 738-750.
5. K. Mathias, D. Whitley : Genetic operators, the fitness landscape and the traveling salesman problem. Proc. Parallel Problem Solving from Nature II, (1992) 219-228.
6. R. Yang : Solving Large Traveling Salesman Problems with Small Populations. Genetic Algorithms in Engineering Systems: Innov's and App's, (1997) 157-162.
7. M. Yamamura, I. Ono, S. Kobayashi : Character-preserving genetic algorithms for traveling salesman problem. Jour. of Japan Society for AI, vol. 6, (1992) 1049-1059.
8. L. Yang, D.A. Stacey : Solving the Traveling Salesman Problem Using the Enhanced Ge-netic Algorithm. LNCS, Vol.2056, Springer-Verlag, 307-316.
9. K.D. Boese : Cost Versus Distance In the Traveling Salesman Problem. Tech. Rep. TR-950018, UCLA CS Department, (1995).
10. D. Whitley, T. Starkweather, D. Fuquay : Scheduling problems and traveling salesman: the genetic edge recombination and operator. Proc. 3rd Int. Conf. GA and their Applications, (1989) 133-140. 441-456.
11. P. Moscato : On Genetic Crossover Operators for Relative Order Preservation. Caltech Concurrent Computation Program, C3P Report 778, (1989).
12. B. Freisleben, P. Merz : A Genetic Local Search Algorithm for Solving Symmetric and Asymmetric Traveling Salesmen Problems. IEEE Int. Conf. on EC, (1996) 616-621.
13. TSPLIB. Web Site, [http://www.iwr.uni-heidelberg.de/iwr/comopt/ soft/ TSPLIB95/TSPLIB.html](http://www.iwr.uni-heidelberg.de/iwr/comopt/soft/TSPLIB95/TSPLIB.html)